Deviations from SRS, SAD and Test Plan - Hostel Management System

1. Introduction

1.1 Purpose

This document lists the deviations between the submitted SRS, SAD and Software Test Plan and the system as built and tested, as required by Mini-Project Deliverables Part-2 (item 2.i). Each deviation gives what was planned, what was done instead, and its impact.

1.2 Scope

It covers the requirements, the design, the testing approach and the project process.

1.3 References

- Software Requirements Specification (SRS) - Hostel Management System
- Software Architecture and Design Specification (SAD) - Hostel Management System
- Software Test Plan (STP) - Hostel Management System

2. Deviations from the SRS

None. All 19 functional, 6 non-functional and 6 security requirements are implemented, and nothing outside the SRS scope (fee payment, attendance, visitor management, integration with other college systems) was added.

3. Deviations from the SAD

#	Deviation	Planned	Actual	Impact
1	Student landing page	Students land on their own home page after login.	Students land on Room availability; their profile is one click away in the navigation bar.	Minor navigation change; no requirement affected.
2	Hostel fields	Hostel recorded with type and warden (STP UT_HOS_01 data).	Hostel has name, code, total capacity, address and notes; the warden can be written in notes.	UT_HOS_01 test data revised.
3	Logging to a file	logs/hms.log with INFO, WARNING and ERROR levels (SAD 3.4).	Services log events with IDs only, but no file handler is configured, so events go to the console.	ST_SEC_01 log search step does not apply (defect D-04).
4	Deployment hardening	manage.py check --deploy before the demo; secure cookies on HTTPS (STP 5.1).	Secure cookies are enabled when DEBUG=False. HSTS and SSL redirect are not set because the system is not served over HTTPS.	Two configuration warnings (defect D-05).

4. Deviations from the Test Plan

#	Deviation	Planned	Actual	Impact
5	System tests automated	Manual browser runs, then Selenium (STP 5).	System tests drive the real URLs, views, templates and database through the Django test client and run in CI on every push.	Repeatable on every change; browser rendering is checked in the demo.
6	Performance test tool	Locust, 20 users for 5 minutes (ST_PERF_01).	In-process load test: 160 requests over 4 pages on 500 students and 100 rooms.	No concurrent users; result (21 ms at the 90th percentile) is far inside the 3 s target.
7	Restart test	Stop and restart the server (ST_REL_01).	One process writes records to a file database and exits; a new process opens it and checks them.	Same check without a web server.
8	Usability walk-through	Three students who did not build HMS (ST_UX_01).	Not yet executed; planned for the UAT demo.	HMS-NF-004 partly validated until then.
9	Dashboard comparison	Compare with DB Browser for SQLite (ST_DASH_01).	An automated test compares each value on the page with a direct database count.	None.
10	Test case changes	58 designed test cases.	15 cases revised to match the implementation and 7 added (65 in total); see the updated test case document.	Better coverage of the built system.

5. Process Deviations

#	Deviation	Planned	Actual	Impact
11	Third sprint	Development in two sprints (Deliverables Part-2).	Room allocation, modify, cancel and history (HMS-F-010 to F-014, F-018) and check-in / check-out (F-015, F-016) were completed in Sprint 3, as Deliverables Part-3 allows for spill-over.	None: all 19 functional requirements are built and tested.
12	Demo video	One video for Sprint 1 and one for Sprint 2 (Deliverables Part-2).	One combined video covering both sprints.	Same content in one file.